

M2 Data Science – M2 Intelligence Artificielle

Data Mining

Miner des graphes
et des processus

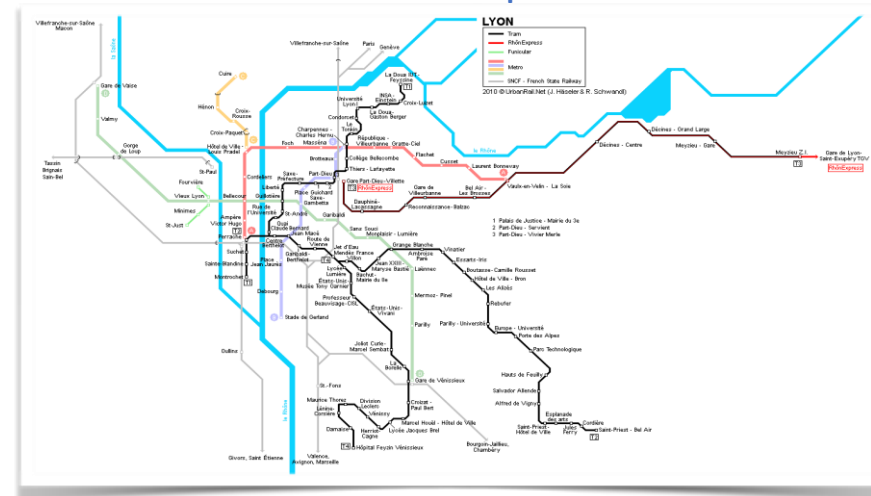


Exemples de réseaux dans le monde réel

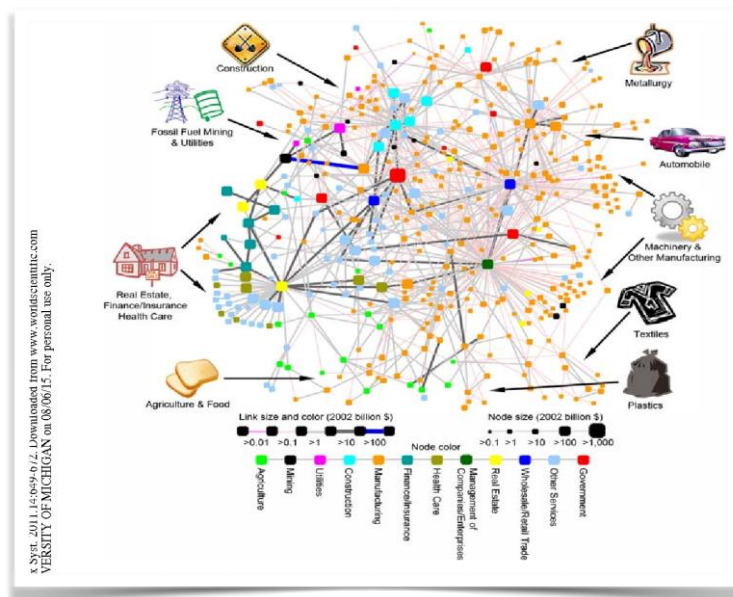
Traffic aérien



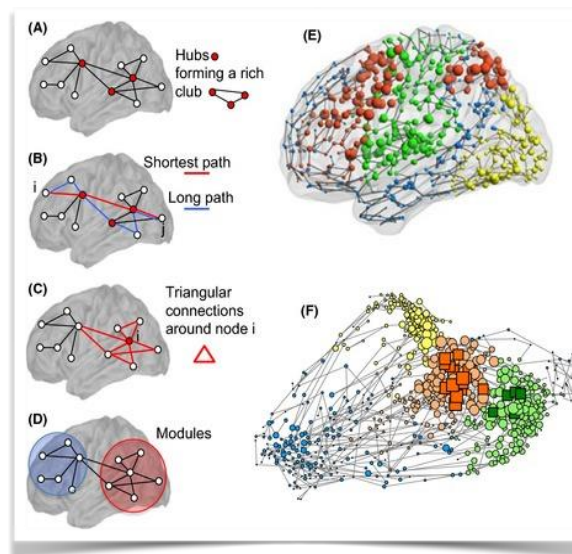
Transports



Transactions
marchandes



Données
médicales

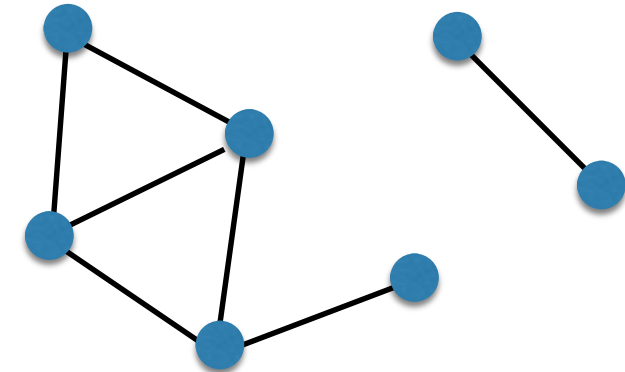


Et bien d'autres...

Définitions: Réseaux et graphes

- Les **réseaux** (*networks*), réfèrent à des systèmes réels concrets
 - Réseaux internet
 - Réseaux sociaux
 - Réseaux autoroutiers
 - Etc.
 - Vocabulaire: Réseau (*network*), nœud (*node*), lien (*link*)
- Les **graphes** (*graphs*), sont la représentation mathématique des réseaux
 - Vocabulaire: Graphe (*graph*), *vertex*, *edge*

=> La plupart du temps les termes sont inter-changeables



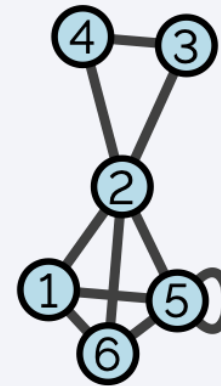
Notations mathématiques

- $G = (V, E)$: un graphe, avec:
 - V : l'ensemble de ses *vertices* (nœuds)
 - E : l'ensemble de ses *edges* (liens)
 - $u \in V$: un vertex/nœud particulier
 - $(u, v) \in E$: un edge/liens particulier

Exemple

Network - Graph notation

Graph



Graph notation

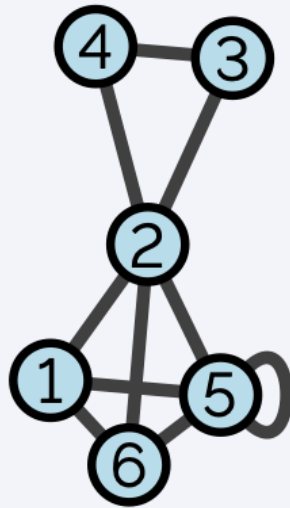
$$G = (V, E)$$

$$V = \{1, 2, 3, 4, 5, 6\}$$

$$E = \{(1, 2), (1, 6), (1, 5), (2, 4), (2, 3), (2, 5), (2, 6), (6, 5), (5, 5), (4, 3)\}$$

Matrices d'adjacences

Graph

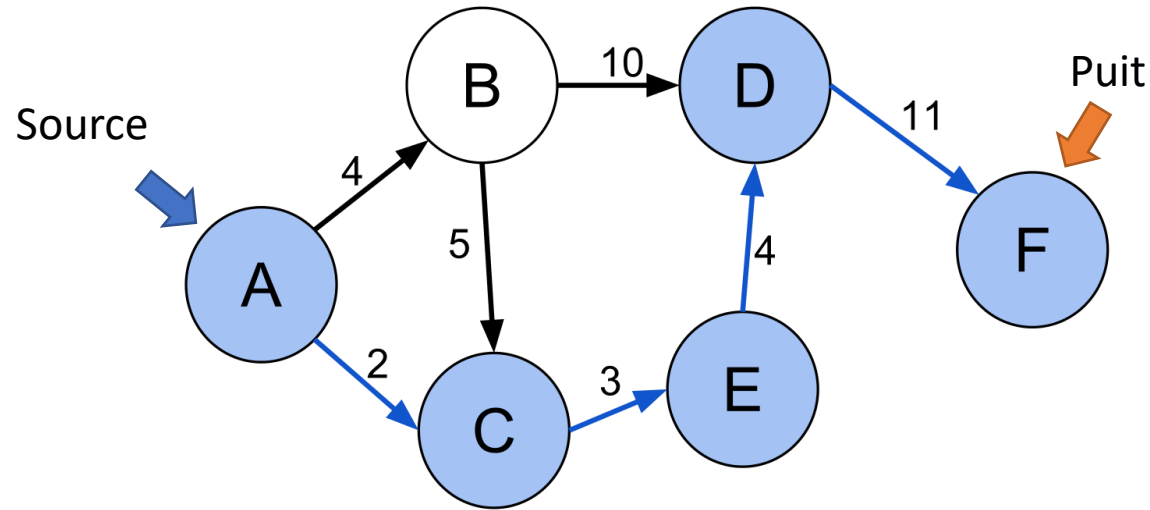


A - Adjacency Mat.

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Notez que cette matrice est symétrique

Variantes: directions et/ou poids



	0	4	2	0	0	0
	0	0	5	10	0	0
	0	0	0	0	3	0
	0	0	0	0	0	11
	0	0	0	4	0	0
Puit	0	0	0	0	0	0
Source						

- Matrice d'adjacence non-symétrique
- Poids des liens dans la matrice
 - $w_{u,v}$, le poids du lien $(u, v) \in E$
 - s_u , la « force » de u , la somme des poids des liens adjacents à u , $s_u = \sum_v w_{u,v}$
- Présence de « sources » et de « puits »

Compter les nœuds et les liens

- **N/n** : la « taille » d'un graphe, càd le nombre de ses nœuds $|V|$
- **L/m** : le nombre de liens $|E|$ d'un graphe
- **L_{max}** : le nombre maximum de liens possibles dans un graphe de taille N
 - **Graphe orienté**: $L_{max} = N(N - 1)$
 - **Graphe non-orienté**: $L_{max} = \frac{N(N-1)}{2}$

Extraire des graphes

Place à la pratique !

- Sur [kaggle](#), téléchargez l'un des jeux de données suivants:
 1. [Airports, Airlines, Planes & Routes](#)
 2. [Superstore Sales Data](#)
 3. [League of Legends Ranked Matches](#)
 4. [Korean Drama from 2015-2023](#)
 5. [NBA games data](#)
 6. [Diseases and their Symptoms](#)
 7. [Banking Transactions Dataset](#)
- Vérifiez et nettoyez vos données !
- Discutez sur quels types de graphes vous pouvez extraire de ces jeux de données
 - Que peuvent représenter les nœuds de votre graphe?
 - Que peuvent représenter les liens entre les nœuds de votre graphe ?
- Discutez des informations que vous pouvez extraire de ces graphes
- Faites des tests et discutez de vos résultats

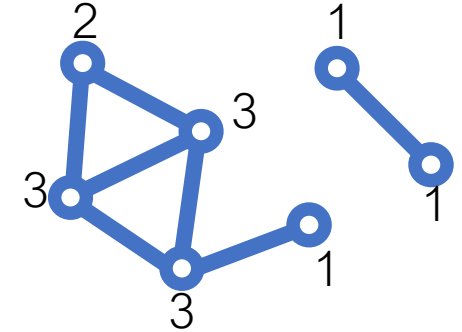


Décrire un graphe

Degrés et voisinage d'un nœud

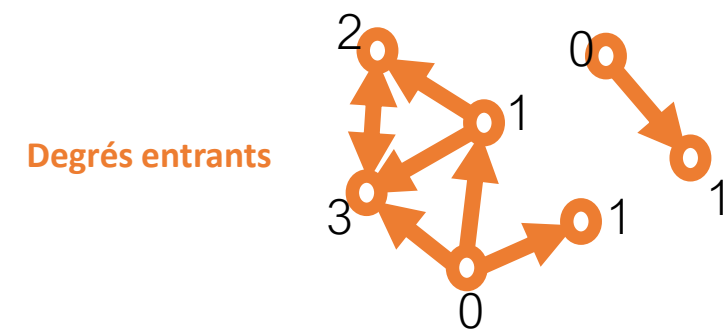
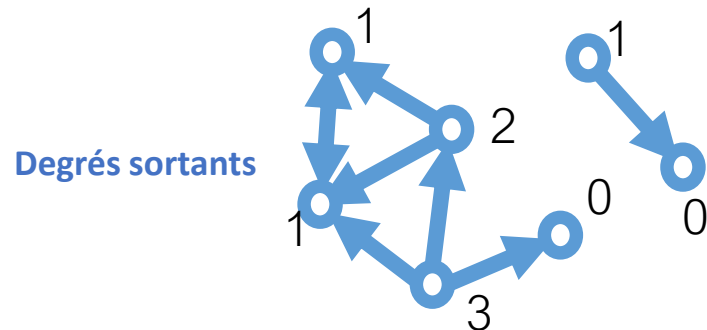
Graphes non-orientés:

- N_u : le «voisinage» de u , l'ensemble des nœuds partageant un lien avec u
- δ_u : le « degré » de u , le nombre $|N_u|$ des voisins de u



Graphe orientés:

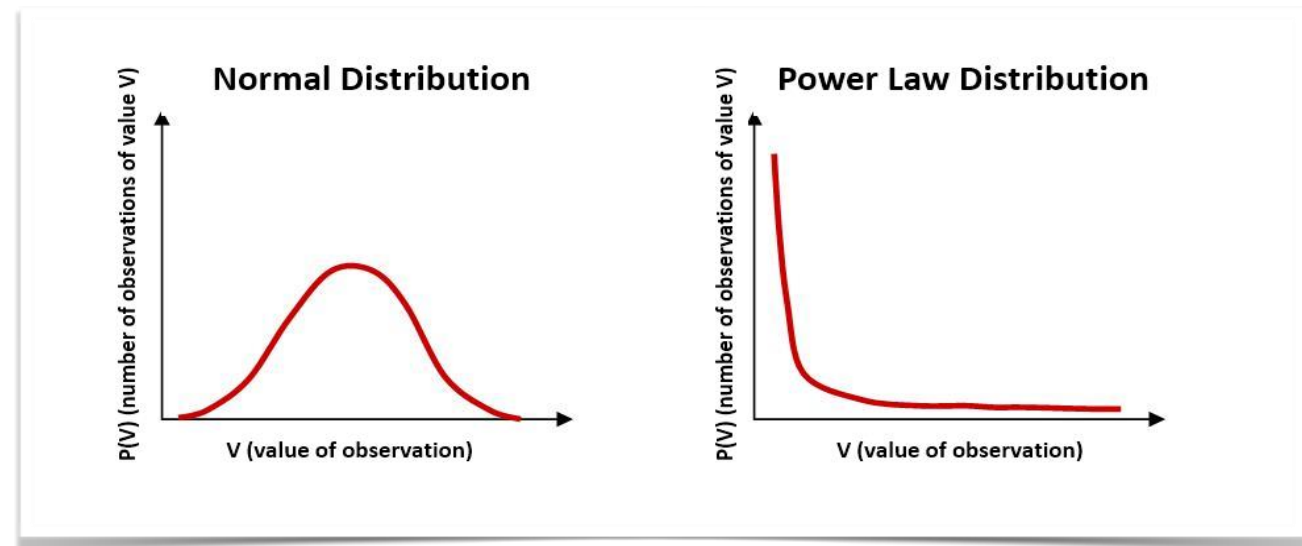
- N_u^{out} : les « successeurs » de u , l'ensemble des nœuds $v \in V$ tel qu'il existe un lien $(u, v) \in E$
- N_u^{in} : les « prédécesseurs » de u , l'ensemble des nœuds $v \in V$ tel qu'il existe un lien $(v, u) \in E$
- δ_u^{out} : le « degré sortant » de u , le nombre $|N_u^{out}|$ de successeurs de u
- δ_u^{in} : le « degré entrant » de u , le nombre $|N_u^{in}|$ de prédécesseurs de u



Décrire un graphe

Densité

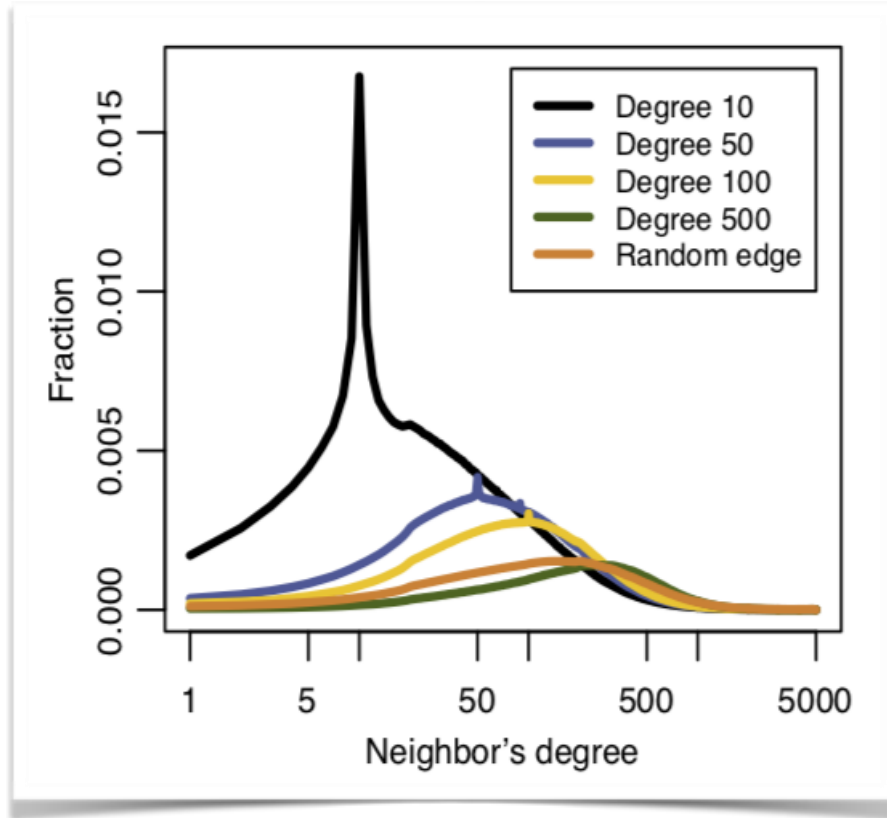
- $d / d(G)$: la proportion du nombre de liens existants dans G par rapport au nombre max de liens possible
 - $d = \frac{L}{L_{max}}$
- Dans les graphes issus de données réelles, les degrés des nœuds suivent souvent une loi de puissance



Décrire un graphe

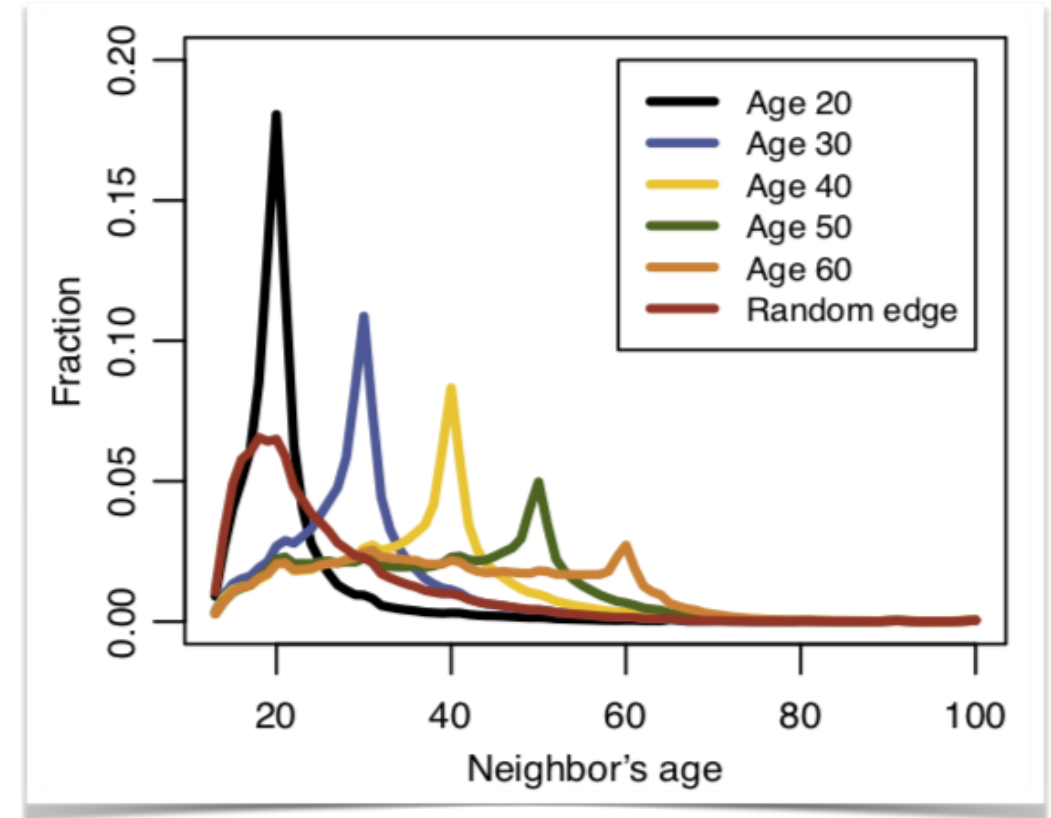
Analyser un nœud par rapport à son voisinage

Rapport entre le degré d'un nœud et le degré de ses voisins



Ex. Mes amis ont généralement le même nombre d'amis que moi

Rapport entre une variable d'un nœud et la même variable chez les nœuds voisins

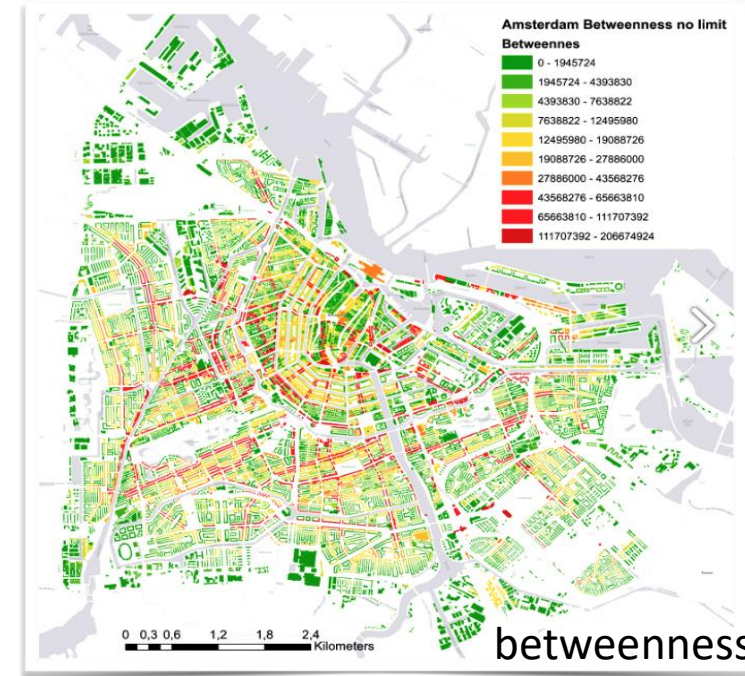
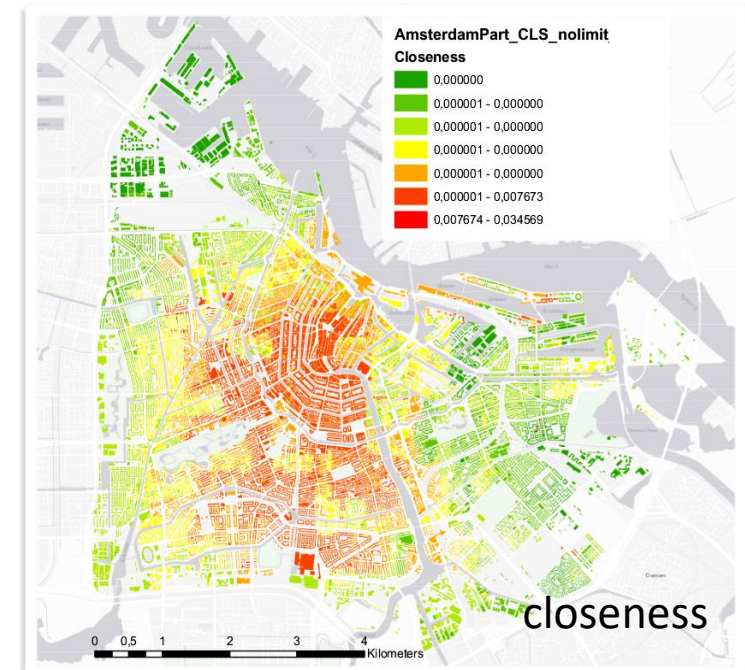


Ex. Chaque personne est généralement amie avec des personnes du même âge

Décrire un graphe

Repérer les nœuds importants

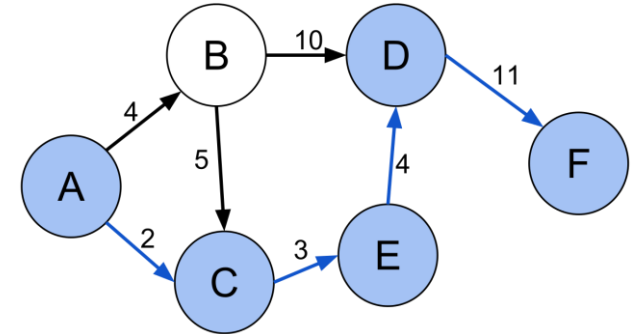
- Le **degré** est souvent suffisant pour trouver les nœuds les plus importants
 - Mais vite limité dans sur des données réelles (ex. faux utilisateurs sur facebook)
- La **closeness**, mesure à quel point un nœud est proche des autres nœuds
 - $closeness(u) = \frac{N-1}{\sum_{v \in V \setminus u} l_{u,v}}$, avec $l_{u,v}$ la taille du chemin le plus court entre u et v
 - Variante, l'**hamonic centrality**: $harmonic(u) = \frac{1}{N-1} \sum_{v \in V \setminus u} \frac{1}{l_{u,v}}$
- À l'inverse, la **farness**, mesure à quel point un nœud est éloigné des autres nœuds
 - $farness(u) = \frac{1}{N-1} \sum_{v \in V \setminus u} l_{u,v}$
- La **betweenness**, mesure à quel point un nœud sert de « pont » dans le graphe
 - $betweenness(u) = \sum_{s \neq u \neq t \in V} \frac{\sigma_{s,t}(v)}{\sigma_{s,t}}$, avec :
 - $\sigma_{s,t}$, le nombre de chemin les plus courts entre s et t
 - et $\sigma_{s,t}(v)$, le nombre de chemin les plus courts entre s et t passant par v
 - À noter: fonctionne aussi avec des liens (*edges*)



Décrire un graphe

Repérer les chemins les plus courts

- Un **chemin** (*path*), est une succession de nœuds adjacent distincts
 - Ex. $\langle A, B, D, F \rangle$ est un chemin
- La **taille** d'un chemin (*path length*), est la somme des poids de chaque liens du chemin
 - Ex. la taille du chemin $\langle A, B, D, F \rangle$ est de $4 + 10 + 11 = 25$
 - Si il n'y a pas de poids sur les liens, alors c'est le nombre de nœuds rencontrés (Ex. 3, pour le chemin $\langle A, B, D, F \rangle$)
- Le **chemin le plus court** (*shortest path*) est le chemin de taille minimal reliant deux nœud u et v
 - Souvent non unique
 - La **distance** $l_{u,v}$ est la taille du chemin le plus court entre u et v
- De nombreux algorithmes pour trouver les chemins les plus courts dans un graphe
 - Dijkstra
 - Floyd-Warshall
 - A*
 - Ant Colony Optimization (ACO) <- mon préféré
 - Et bien d'autres...



Propriété intéressante:

la distance moyenne d'un graphe:

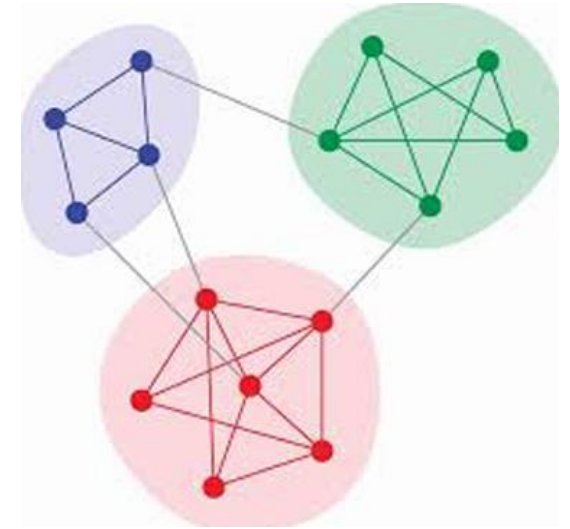
$$\langle l \rangle = \frac{1}{N(N-1)} \sum_{u \neq v} l_{u,v}$$

est très souvent aux alentours de 5 ou 6

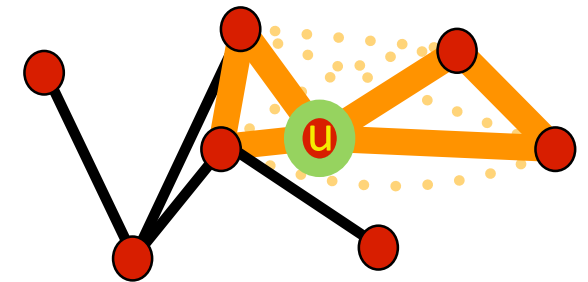
Ex. vous êtes, en moyenne, à 5 ou 6 amis de n'importe qui dans le monde

Décrire un graphe

Repérer les groupes / communautés



- Un **sous-graphe** (*subgraph*), noté $H(W)$, est un sous-ensemble de nœuds $W \subset V$ d'un graphe $G = (V, E)$, associé au sous-ensemble $E' \subset E$ des liens connectant ces nœuds entre eux dans G , tel que:
 - $H(W) = (W, E'), W \subset V, (u, v) \in E' \Leftrightarrow u, v \in W \wedge (u, v) \in E$
- Une **clique** est un sous-graphe de densité $d = 1$
- Un **triangle** est une clique de taille 3
- Une **composante connectée** (*connected component*) d'un graphe est un sous-graphe dont chaque paires de nœuds sont connectés entre eux par un chemin, et qui n'est connecté à aucun nœud supplémentaire dans le super-graphe
- Le **coefficient de regroupement d'un nœud** (*node clustering coefficient*), noté C_u , est calculé comme:
 - La densité du sous-graphe induit par le voisinage de u : $C_u = \frac{d(H(N_u))}{\delta_u}$
 - La fraction de tous les triangles possibles dans $H(N_u)$: $C_u = \frac{\delta_u}{\delta_u^{max}}$



Repérer des groupes/communautés

Place à la pratique !

- Reprenez les jeux de données téléchargés précédemment:
 1. [Airports, Airlines, Planes & Routes](#)
 2. [Superstore Sales Data](#)
 3. [League of Legends Ranked Matches](#)
 4. [Korean Drama from 2015-2023](#)
 5. [NBA games data](#)
 6. [Diseases and their Symptoms](#)
 7. [Banking Transactions Dataset](#)
- Tester les notions que l'on vient d'aborder
- Discutez sur comment repérer des groupes (sous-graphes) dans vos graphes
- Faites des tests et discutez de vos résultats

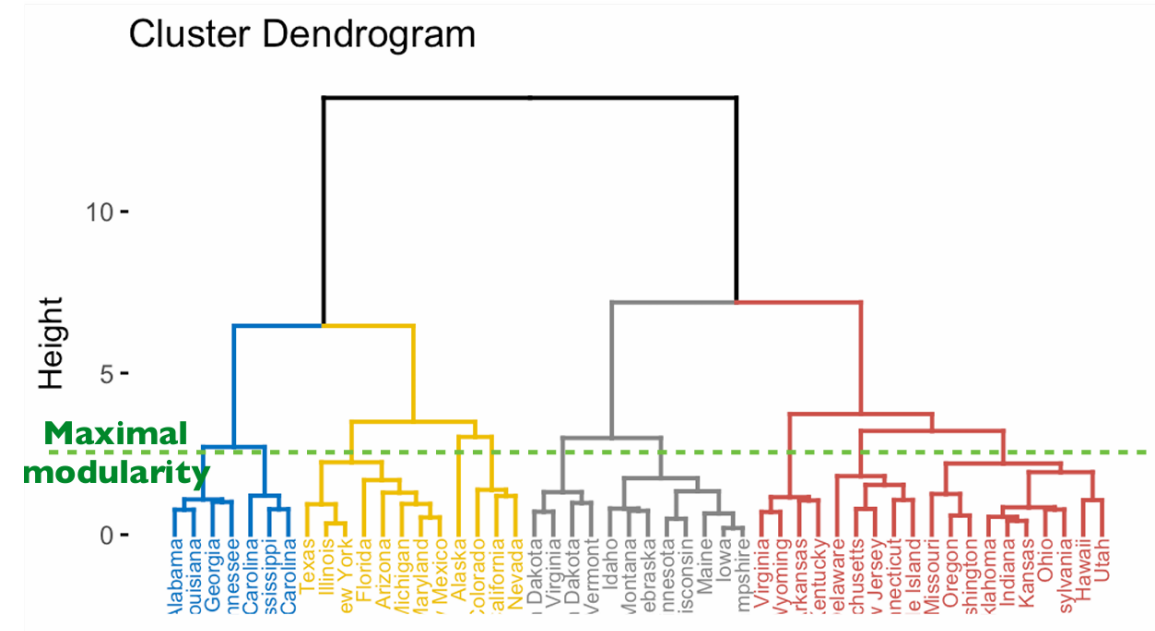


Repérer les groupes / communautés

Approche par division: Girvan & Newman

Algorithme:

1. Calculer la betweenness de chaque lien / edge
2. Supprimer le lien avec la betweenness la plus haute
3. Répéter jusqu'à ce que tous les liens aient été supprimé
 - On obtient ainsi un dendrogramme →
 - On cherche à couper ce dendrogramme de manière à maximiser la **modularité** Q



$$Q = \frac{1}{2m} \sum_{(u,v) \in V} \left[E_{(u,v) \in V} - \frac{\sigma_u \sigma_v}{2m} \right] \delta(c_u, c_v)$$

Additionne pour toutes les paires de nœuds (around the sum)
 Probabilité qu'il y ait un lien entre eux (around the fraction)
 Si ils sont dans la même communauté (around the delta)

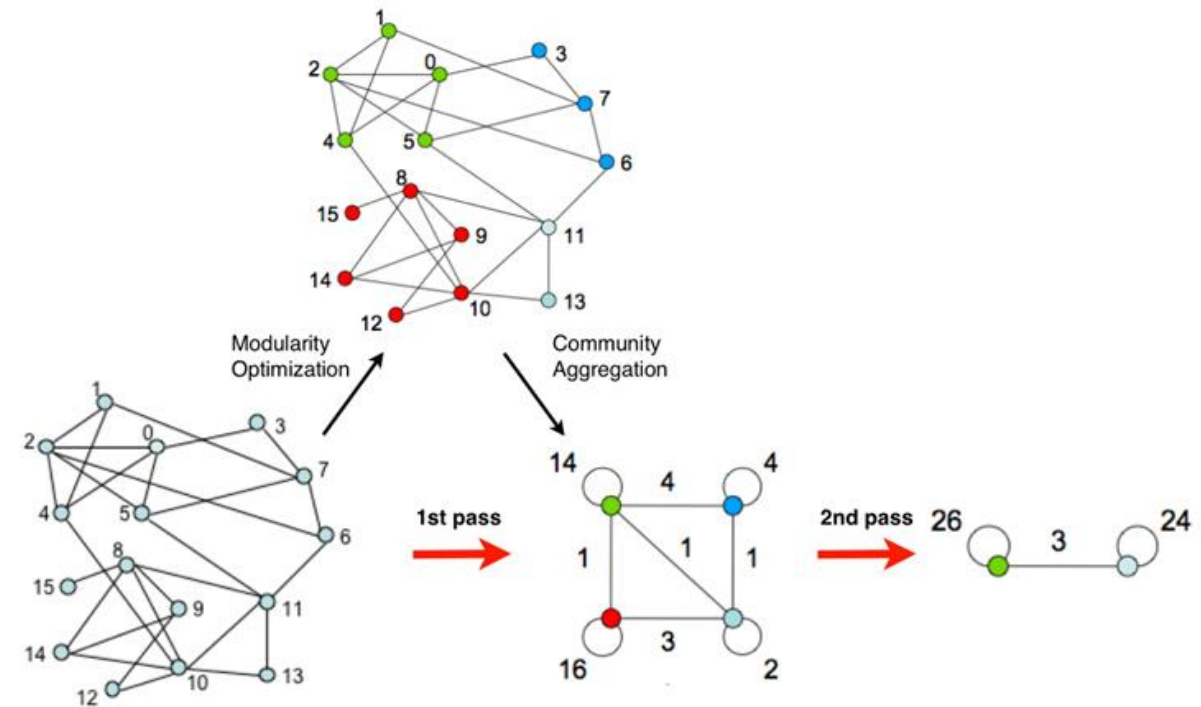
Si il y a un lien entre eux (under the first bracket)
 Si ils sont dans la même communauté (under the second bracket)

Repérer les groupes / communautés

L'algorithme de Louvain

Algorithme:

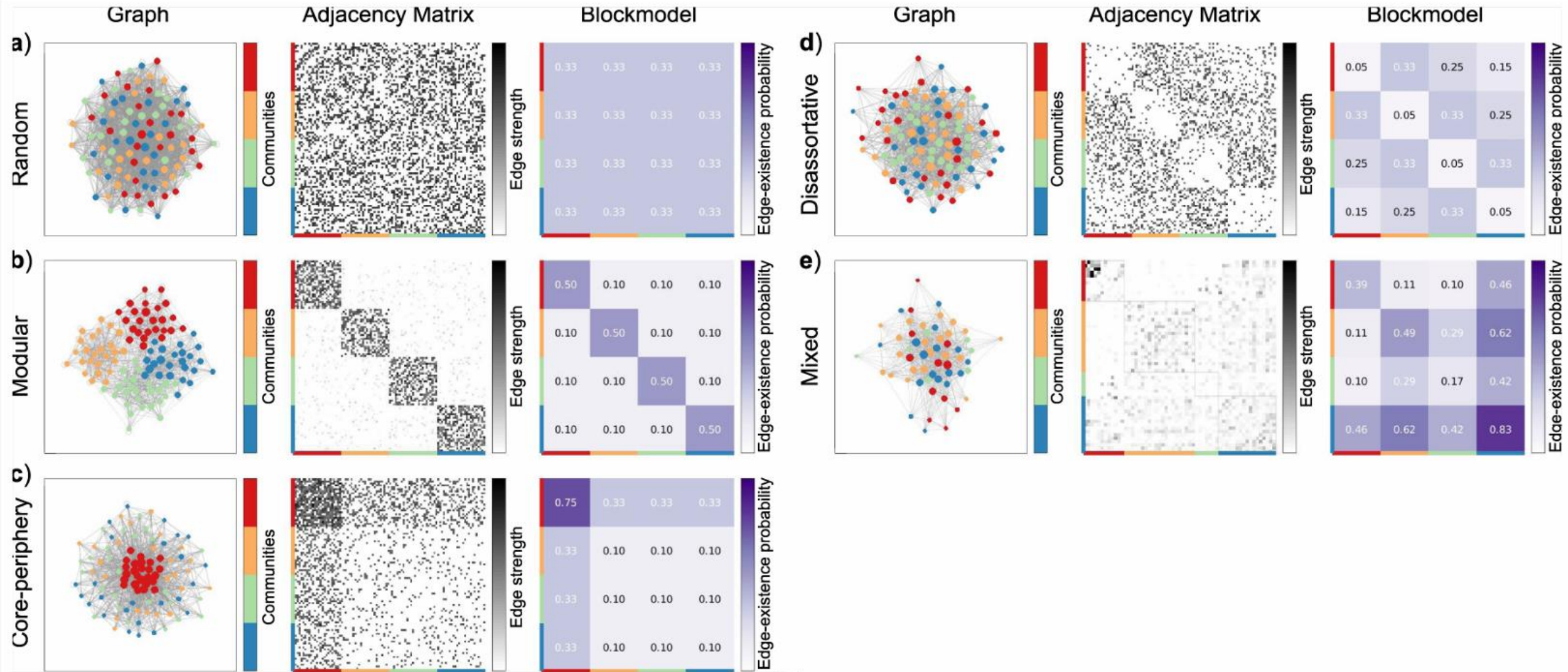
1. Chaque nœud commence comme appartenant sa propre communauté
2. Répéter jusqu'à « convergence »:
 1. Pour chaque nœud $u \in V$:
 1. Pour chaque voisin $v \in N_u$:
 1. Si ajouter u à la communauté de v améliore la modularité: faite le
3. Une fois que les résultats ont convergé, créer un graphe induit (*induced network*)
 - Chaque communauté devient un nœud
 - Le poids des liens entre chaque nœud/communauté est la somme des liens existants entre les communauté avant de les regrouper





Repérer les groupes / communautés

Stochastic Block Models (SBM)



Plus de détails ici:

https://cazabetremy.fr/Teaching/data_class/Graphs_ML.pdf

Repérer des groupes/communautés

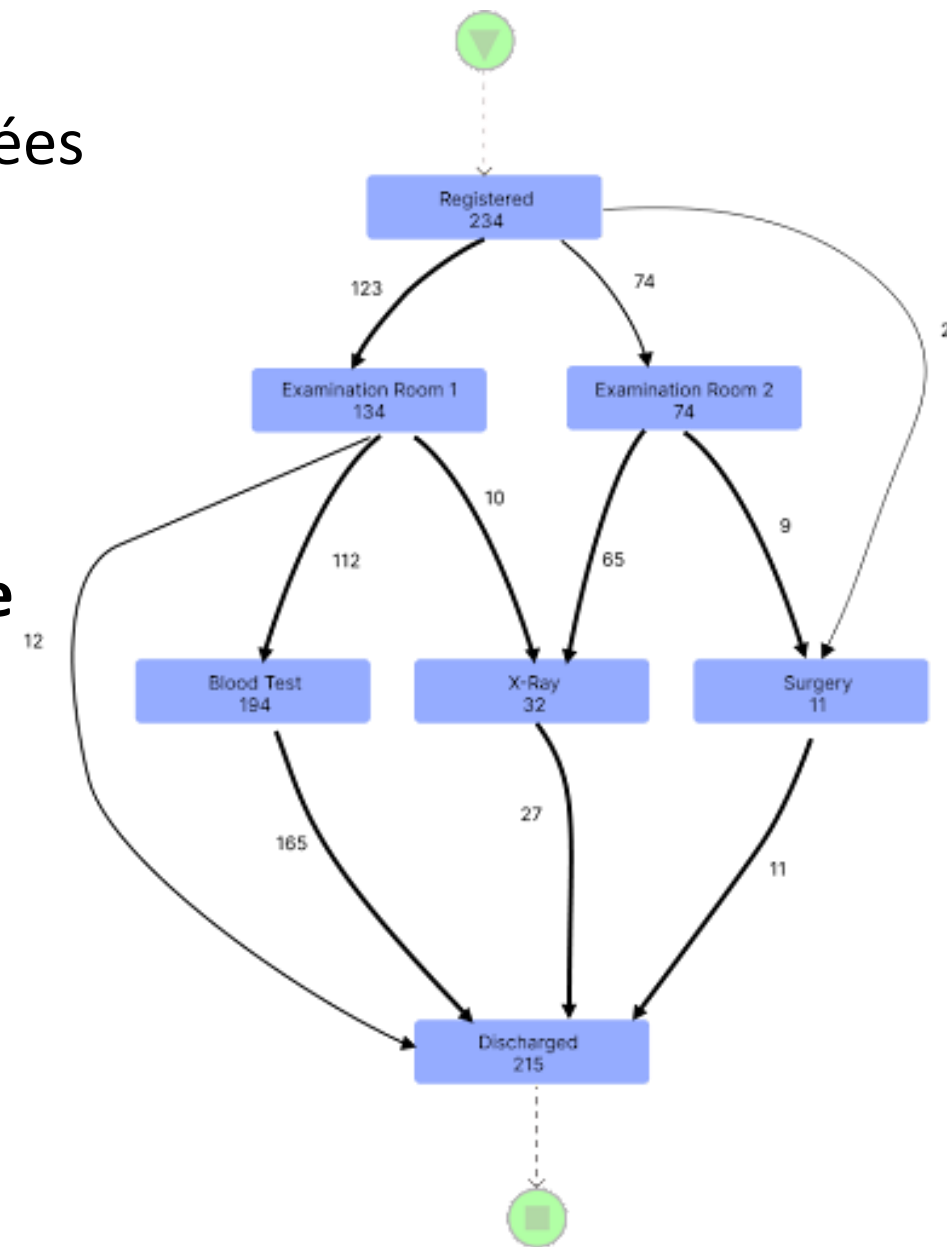
Place à la pratique !

- Reprenez les jeux de données téléchargés précédemment:
 1. [Airports, Airlines, Planes & Routes](#)
 2. [Superstore Sales Data](#)
 3. [League of Legends Ranked Matches](#)
 4. [Korean Drama from 2015-2023](#)
 5. [NBA games data](#)
 6. [Diseases and their Symptoms](#)
 7. [Banking Transactions Dataset](#)
- Tester les algorithmes que l'on vient de voir
- Discutez des groupes trouvés et des informations que vous pouvez en tirer
- Faites des tests et discutez de vos résultats



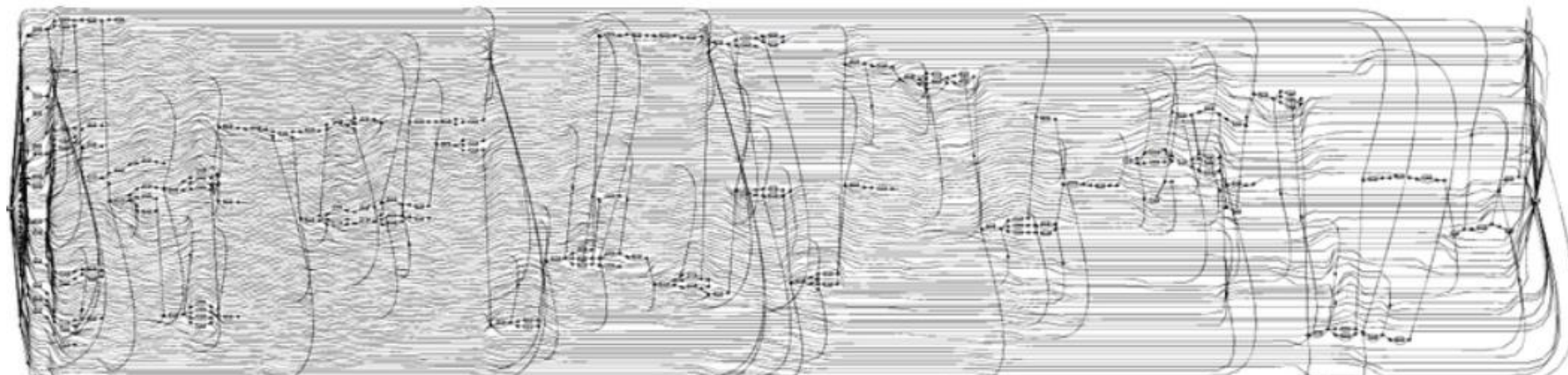
Définitions: Processus

- Un **processus** est une succession de **tâches** effectuées par différents **agents**
 - Ex. activité d'une entreprise ou déplacements de marchandises
 - On peut aussi parler de « séquence » temporelle, d'évènements, d'activités, etc.
- Généralement représenté sous forme d'un **graphe orienté et pondéré**
 - Chaque nœud représente une tâche
 - On peut y associer les agents ou le temps passé
 - Chaque lien (u, v) correspond à l'enchaînement d'une tâche u par une tâche v
 - Ce lien est pondéré par le nombre d'occurrence, ou la fréquence, de cet enchaînement de tâches
 - Deux nœuds « spéciaux »: début et fin de processus



Process Mining

- **Problème:** les données réelles c'est le bordel
 - On obtient souvent ce genre de graphe, dit *spaghetti-like*



- L'objectif de ***process mining*** est d'extraire, à partir d'***events logs*** ou de **traces**, des processus significatifs permettant de décrire l'activité d'un milieu

Extraire des processus

Place à la pratique !

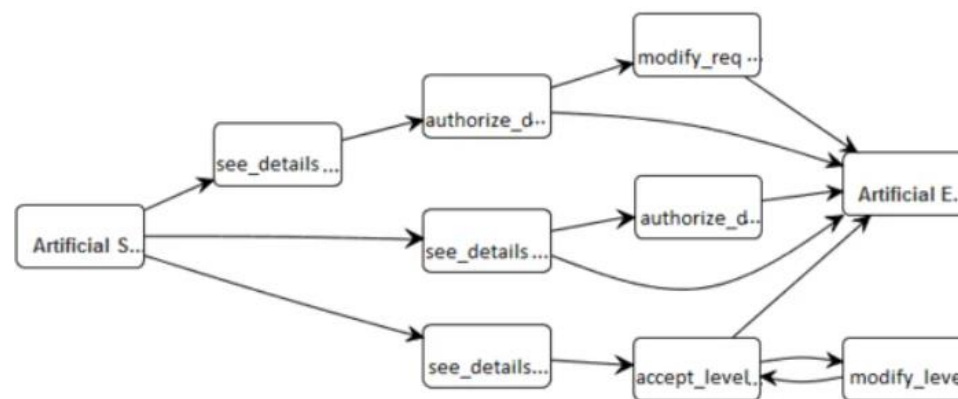
- Téléchargez le jeu de données suivant:
 1. [Event logs for process mining](#)
- Représentez les activités de ces event logs sous forme de graphes
- Discutez sur comment extraire des sous-processus significants
 - Suggestion de librairie: [pm4py](#)
 - Attention: il peut être nécessaire de mettre vos données sous un format spécifique
- Faites des tests et discutez de vos résultats



Process Mining

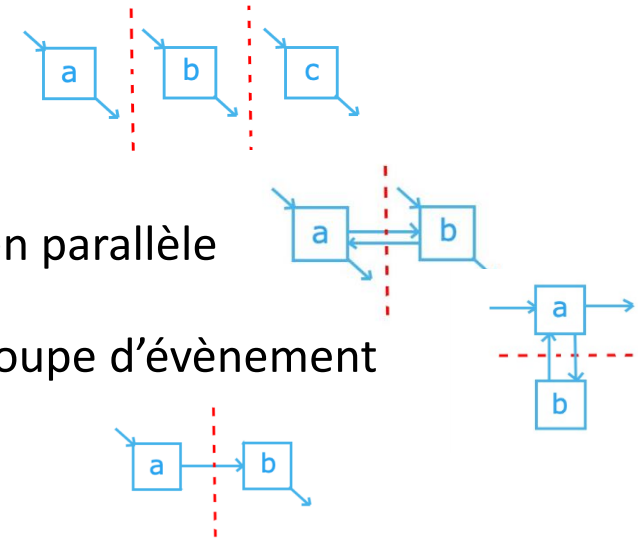
Heuristic Miner

- Se base sur l'heuristique que les événements les plus co-dépendants forment les processus sous-jacents les plus révélateurs d'un processus global
- Calcule un degré de dépendance $u \Rightarrow_W v$ pour chaque couple d'évènements u et v dans un ensemble de traces W :
 - $u \Rightarrow_W v = \frac{|u >_W v| - |v >_W u|}{|u >_W v| + |v >_W u| + 1}$, avec $u >_W v$ l'ensemble des liens allant de u vers v
 - On obtient une matrice de dépendance entre chaque nœud du graphe
- On supprime les liens entre les nœuds dont la dépendance est inférieure à λ
 - Si on un nœud se retrouve isolé, on le supprime aussi
- On obtient un graphe simplifié →
- Problème: Vite limité sur de très grands graphes



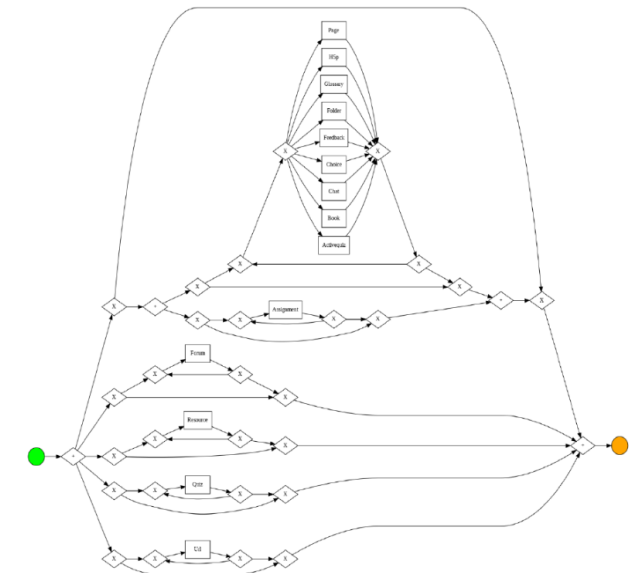
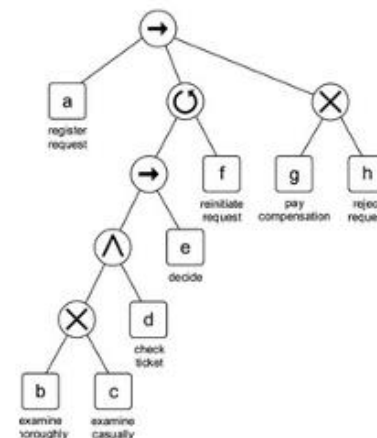
Process Mining Inductive Miner

- Recherche à identifier des endroits où « couper » le processus:
 - OU exclusifs: sépare les groupes d'évènements sans aucun lien entre eux
 - Parallèle: sépare les groupes d'évènements interdépendants s'exécutant en parallèle
 - Redo: sépare les groupes d'évènements revenant toujours sur un autre groupe d'évènement
 - Séquence: sépare les groupes d'évènements se succédant directement



- Retourne un arbre d'activité ou un graphe simplifié

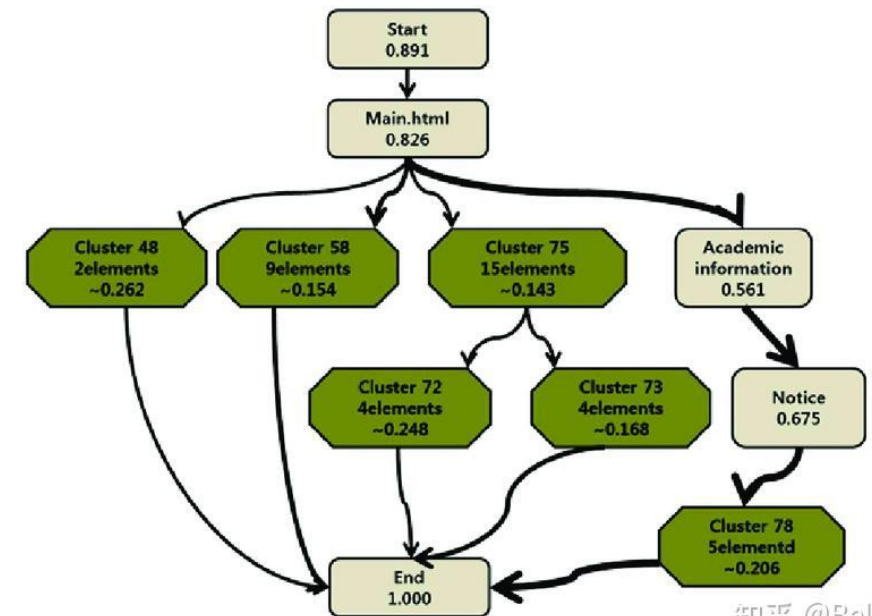
- Fonctionne bien sur de grands graphes
- plusieurs variantes existent
- Mais résultat moins intuitif à analyser et qui peut vite être illisible avec beaucoup de nœuds



Process Mining

Fuzzy Miner

- Calcule un degré de « signifiante » pour chaque nœud et lien du graphe
- Regroupe ou supprime les nœuds et liens selon les trois règles suivantes:
 - Les nœuds et arcs dépassant un seuil λ de signifiante sont préservés
 - Les nœuds et arcs dont le degré de signifiante est inférieur à λ , mais fortement « corrélés » entre eux, sont regroupés dans un nouveau nœud (*cluster*)
 - Les nœuds et arcs peu « signifiant » et peu « corrélés » sont supprimés
- Ce processus peut être répété plusieurs fois
- On obtient ainsi un graphe simplifié



Petit point sur le Process Mining avant de finir

- Le Process Mining est un domaine très orienté « Business » :
 - Premier objectif: découvrir des sous-processus optimaux ([taylorisme](#))
 - À lire sur le sujet: ["Un taylorisme augmenté" \(2025\)](#), de Juan Sebastián Carbonell
 - Second objectif: contrôler que les activités suivent bien un processus pré-établit (enjeux coercitifs sur le travail)
- « Erreur » du Process Mining :
 - **Réinventer la roue:** un processus peut être vu comme un graphe pourtant le domaine du Process Mining n'a pas réexploiter des années de recherche sur la théorie des graphes
- Leçon à en tirer:
 - Face à des données particulières, essayer d'abord de les ramener à des concepts existants

Place à l'action !



Votre Mission:

- Reprendre votre code là vous l'aviez laissé
 - Lien initial: <https://github.com/a-t-richard/20262027-UE-DataMining-Project>
- Utiliser les nouvelles notions et méthodes vues pendant ce cours
- Trouver ce qu'il se cache dans ces données !